МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (НИ ТГУ)
Научно-образовательный центр «Высшая ИТ школа»

КУРСОВАЯ РАБОТА

Разработка iOS-приложения для адаптивного планирования задач с учётом состояния пользователя

Коротков Глеб Дмитриевич

По направлению подготовки 09.03.04 Программная инженерия

«Программная инженерия»

Руководитель:
должность, учёная степень, учёное звание
_____ / Иванова Л. С.
«____» _____ 2026 г.
Автор работы:
студент группы _____
_____ / Коротков Г. Д.
«____» _____ 2026 г.

Томск – 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ
1 Анализ предметной области
1.1 Особенности планирования задач с учётом состояния пользователя
1.2 Анализ приложений-аналогов
1.3 Нефункциональные требования
1.4 Функциональные возможности приложения
1.4.1 Текстовые сценарии ключевых вариантов использования 0
1.5 Модель предметной области
2 Проектирование
2.1 Выбор платформы и средств разработки
2.1.1 Выбор UI–фреймворка
2.1.2 Выбор инструментов разработки
2.2 Архитектурный подход
2.2.1 Распределение ответственности
2.2.2 Обоснование выбора MVVM-C
2.2.3 Преимущества выбранного подхода
2.3 Хранение данных и системные интеграции
2.4 Проектирование пользовательского интерфейса
3 Реализация приложения.....26
3.1 Учётная запись, профиль и сессия
3.2 Календарь и управление элементами
3.3 Автоматическое планирование и граница 23:00
3.4 Уведомления и дневник
3.5 Фокус-сессия и Live Activity
3.6 Анализ дня, статистика и достижения
3.7 Пользовательское исследование восприятия интерфейса
ЗАКЛЮЧЕНИЕ
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....47
ПРИЛОЖЕНИЕ А
ПРИЛОЖЕНИЕ Б

ВВЕДЕНИЕ

План на день редко остаётся неизменным. Новые задачи появляются уже после того, как расписание составлено, отдельные дела занимают больше времени, чем ожидалось, а работоспособность пользователя зависит не только от количества свободных часов. На способность сохранять внимание и выполнять сложные задачи также влияют продолжительность сна, эмоциональное состояние и накопленная нагрузка [1; 2].

Большинство цифровых инструментов рассматривают эти данные раздельно. Календарь хранит события, менеджер задач – перечень дел, а приложения для самонаблюдения – записи о сне или настроении. Пользователю приходится самостоятельно сопоставлять сведения из нескольких источников и решать, какие задачи можно выполнить в доступное время. Поэтому актуальна разработка мобильного приложения, которое объединяет планирование, сведения о состоянии пользователя и рекомендации в одном интерфейсе.

Под адаптивным планированием в работе понимается формирование плана и рекомендаций с учётом календарной занятости, параметров задач, сведений о сне и эмоциональном состоянии пользователя.

Работа выполняется в рамках заказной разработки для частного заказчика, заинтересованного в создании персонального iOS-планировщика. Проектирование опирается на предоставленные технические данные о составе функций, правилах планирования и интеграции с системными средствами iOS. Результатом должна стать версия Luma, подготовленная для передачи заказчику.

Цель курсовой работы – разработать iOS-приложение Luma для адаптивного планирования задач с учётом состояния пользователя.

Для достижения поставленной цели необходимо решить следующие задачи:

1. изучить возможности iOS и системных сервисов, применимых при разработке приложения;

2. проанализировать существующие решения и определить их основные недостатки;

3. определить способы устранения выявленных недостатков с учётом технического задания заказчика;

4. спроектировать архитектуру и функциональные возможности приложения;

5. реализовать спроектированные функциональные возможности приложения.

Практическая значимость работы состоит в создании Luma – мобильного приложения, которое объединяет задачи, календарные события, дневники состояния, фокус-сессии и статистику и может быть передано заказчику для практического использования.

1 Анализ предметной области

Исходной точкой проектирования Luma служат технические данные и ограничения, предоставленные заказчиком. На этапе анализа они были сгруппированы по функциональным областям и сопоставлены с пользовательскими сценариями. Анализ приложений-аналогов дополняет постановку задачи, но не изменяет границы разрабатываемого приложения.

1.1 Особенности планирования задач с учётом состояния пользователя

Планирование повседневной деятельности включает не только составление списка дел, но и распределение ограниченного времени между задачами разной продолжительности, сложности и важности. Пользователю необходимо учитывать уже назначенные события, сроки выполнения и возможность переноса отдельных действий. Если задача состоит из нескольких этапов, её также требуется разделить на более короткие и понятные части.

В предметной области можно выделить два основных типа запланированных действий: события и задачи. Событие имеет заранее установленное время начала и окончания, поэтому занимает фиксированный интервал в расписании. Для задачи, напротив, может быть известна продолжительность, но конкретное время выполнения определяется позднее. При формировании плана задачи должны размещаться в промежутках, которые остаются свободными между событиями системного календаря.

Простой список дел не решает эту задачу полностью. Он показывает, что необходимо выполнить, но оставляет пользователю ручное распределение нагрузки. При изменении расписания часть задач приходится переносить, а крупные задачи дополнительно разбивать на этапы. Поэтому планировщик должен работать не только с отдельными записями, но и со связями между событиями, задачами и доступными временными интервалами.

Эффективность планирования зависит и от состояния пользователя. Исследования показывают, что управление временем связано не только с результативностью, но и с субъективным благополучием человека [3]. Продолжительность и качество сна влияют на психоэмоциональное состояние [1], а эмоции могут отражаться на внимании и выполнении текущей деятельности [2]. По этой причине сведения о сне и эмоциональных отметках можно использовать как дополнительный контекст при анализе нагрузки. Приложение при этом не выполняет медицинскую диагностику, а помогает пользователю сопоставлять своё состояние с запланированной деятельностью.

Отдельную часть предметной области образуют фокус-сессии. Они связаны с конкретной задачей и позволяют выделить период, в течение которого пользователь

сосредоточен на её выполнении. Сведения о продолжительности таких сессий и завершённых задачах могут использоваться при формировании статистики. Достижения и уровни дополняют этот процесс визуальной обратной связью и показывают накопленный прогресс.

Основной сценарий работы начинается с добавления задач и событий. Приложение учитывает занятые интервалы системного календаря, после чего распределяет задачи в оставшееся время. Сложная задача предварительно разделяется на подзадачи, каждая из которых получает собственную продолжительность. В течение дня пользователь отмечает выполненные действия, запускает фокус-сессии и при необходимости изменяет план. Дополнительно он может вносить сведения о сне и эмоциональном состоянии, а затем просматривать статистику и рекомендации.

Таким образом, предметная область объединяет календарное планирование, управление задачами и наблюдение за состоянием пользователя. Её основными сущностями являются пользователь, задача, подзадача, событие, запись о сне, эмоциональная отметка, фокус-сессия, достижение и результат анализа. Связи между этими сущностями будут представлены далее с помощью модели предметной области.

1.2 Анализ приложений-аналогов

Для сравнения выбраны TickTick, Structured, Todoist и Fabulous. Первые три продукта относятся к планировщикам задач, а Fabulous ориентирован на формирование привычек и повседневных ритуалов. Сопоставление выполнено по функциям, которые связаны с техническим заданием Luma: управление задачами, календарное представление, планирование времени, фокус-сессии, дневник сна и эмоций. Сведения о возможностях приложений проверены по официальным страницам продуктов и их справочным центрам [4–9].

TickTick поддерживает списки задач, календарные представления, напоминания, фильтры, статистику и встроенный таймер Pomodoro. Structured объединяет задачи в визуальной временной шкале, позволяет импортировать календарь, работать с подзадачами и использовать фокус-таймер. Todoist предоставляет задачи, приоритеты, подзадачи, напоминания и интеграцию с календарём. Fabulous строится вокруг привычек, рутин и фокусирующих практик, поэтому не является полноценным диспетчером задач в смысле технического задания Luma.

Официальные описания не подтверждают, что выбранные продукты одновременно предоставляют адаптивное распределение задач с учётом сложности, важности, сна и эмоционального состояния, а также дневники сна и эмоций, анализ дня и систему достижений в одной модели данных. Поэтому обоснование разработки Luma следует

строить на сочетании функций и правилах их совместной работы, а не на утверждении, что у аналогов отсутствуют отдельные базовые возможности.

Результаты сравнения функциональности представлены в таблице 1.1.

Таблица 1.1 – Сравнение функциональности приложений-аналогов

Критерий	**TickTick**	**Structured**	**Todoist**	**Fabulous**
Управление задачами	Да: задачи, приоритеты, сроки, напоминания	Да: задачи на временной шкале, длительность и сроки	Да: задачи, приоритеты, сроки и напоминания	Частично: привычки и ежедневные ритуалы
Подзадачи	Да	Да	Да	В официальном описании не заявлены
Календарное представление	Да: несколько календарных режимов	Да: временная шкала, недельный и месячный режимы, импорт календаря	Да: календарная интеграция	В официальном описании не заявлено как диспетчер задач
Автоматическое планирование по совокупности важности, сложности и свободного времени	Не подтверждено	Есть средства планирования, но требуемый алгоритм не подтверждён	Не подтверждено	Не подтверждено

Продолжение таблицы 1.1

Критерий	**TickTick**	**Structured**	**Todoist**	**Fabulous**
Встроенная фокус-сессия с таймером	Да: Pomodoro	Да: фокус-таймер	Не подтверждено как встроенная функция	Да: фокусирующие практики; эквивалентность таймеру Luma не подтверждена
Дневник и анализ сна	Не подтверждено	Не подтверждено	Не подтверждено	Есть вечерние и утренние рутины, но требуемый дневник и анализ сна не имеется
Дневник эмоций с интенсивностью и причиной	Не подтверждено	Не подтверждено	Не подтверждено	Не подтверждено
Совместный анализ задач, сна и эмоций	Не подтверждено	Не подтверждено	Не подтверждено	Не подтверждено

Примечание: «не подтверждено» означает, что функция в требуемом виде не найдена в проверенных официальных материалах. Это не равнозначно доказательству полного отсутствия функции во всех версиях продукта.

1.3 Нефункциональные требования

Нефункциональные требования описывают не отдельные действия пользователя, а условия, при которых предусмотренные техническим заданием функции могут считаться пригодными для эксплуатации. Для Luma к таким условиям относятся ограничения платформы, требования к интерфейсу, производительности, сохранности и защите данных, автономной работе и структуре исходного кода. Каждое требование сформулировано так, чтобы его можно было проверить при анализе реализации или во время испытаний приложения.

Luma разрабатывается как нативное приложение для iPhone. Приложение предназначено для устройств, поддерживающих используемые системные средства iOS, включая EventKit, LocalAuthentication, UserNotifications и ActivityKit. Возможности Face ID и Live Activity отображаются только на совместимых устройствах. Их отсутствие, а также отказ в доступе к системному календарю не должны блокировать работу с локальными

задачами и независимыми разделами. Совместимость проверяется запуском приложения на заявленных конфигурациях и выполнением сценариев с недоступными или отозванными разрешениями.

Пользовательский интерфейс выполняется на русском языке и сохраняет единый визуальный стиль во всех разделах. Названия кнопок, полей и экранов должны однозначно отражать их назначение. Пять основных разделов доступны через постоянную нижнюю панель, поэтому переход к каждому из них выполняется одним нажатием. Формы ввода содержат подсказки и обозначение обязательных полей, а отсутствие данных и ошибки сопровождаются понятными сообщениями. Выход из учётной записи и другие значимые действия требуют подтверждения. Читаемость интерфейса проверяется на поддерживаемых размерах экрана: текст и элементы управления не должны перекрываться системными областями, информация не должна передаваться только цветом, а интерактивные элементы должны оставаться различимыми и доступными для нажатия. Дополнительно понятность навигации и форм оценивается в ходе пользовательского исследования.

При работе с локальными данными переключение между разделами, датами и фильтрами должно занимать не более одной секунды. Формирование анализа состояния на контрольном наборе пользовательских данных должно завершаться не более чем за две секунды. Автоматическое планирование, построение статистики и обработка изображения профиля не должны блокировать интерфейс, а обратный отсчёт во время фокус-сессии должен обновляться каждую секунду. Эти ограничения проверяются инструментальными замерами на одном и том же устройстве и воспроизводимом наборе данных.

Требования к надёжности связаны прежде всего с локальным хранением. Задачи, дневниковые записи, фокус-сессии, достижения и сведения учётной записи должны сохраняться после перезапуска приложения. Перед записью проверяются обязательные поля и временные интервалы. Ошибка отдельной операции не должна повреждать ранее сохранённые данные, а повторное обновление экрана – создавать дубликаты задач, достижений или уведомлений. Подзадача сохраняет связь с родительской задачей, данные разных учётных записей не смешиваются, а дата и время обрабатываются с учётом календаря и часового пояса устройства. Проверка включает повторные запуски, неудачное сохранение, работу с несколькими локальными учётными записями и смену часового пояса.

Пароль пользователя не должен храниться в открытом виде. Перед сохранением он преобразуется криптографической хеш-функцией с индивидуальной солью, а данные, необходимые для проверки учётной записи, размещаются в защищённом системном хранилище Keychain. Биометрическая проверка выполняется средствами iOS, поэтому приложение не получает биометрические данные пользователя. После выхода доступ к

прежней сессии возможен только после повторной авторизации. Разрешения на календарь, уведомления и медиатеку запрашиваются непосредственно перед использованием соответствующей функции. Задачи, записи о сне и эмоциях и результаты анализа хранятся локально и не передаются внешним сервисам без отдельного согласия. Эти требования проверяются инспекцией исходного кода и хранилища, а также сценариями входа, выхода, отказа в разрешениях и наблюдением за сетевой активностью.

Основные функции приложения должны оставаться доступными без подключения к Интернету. Регистрация локальной учётной записи, авторизация, работа с задачами, дневником, анализом и статистикой выполняются на устройстве, а музыка для фокус-сессии хранится в ресурсах приложения. Временная недоступность системного календаря не должна препятствовать работе с задачами, созданными внутри Luma. После завершения или отмены фокус-сессии должны останавливаться таймер, воспроизведение музыки и Live Activity; наблюдатели и фоновые операции освобождаются после окончания связанного с ними сценария. Автономность проверяется запуском без сетевого подключения, а завершение ресурсов – профилированием жизненного цикла соответствующих компонентов.

Сопровождаемость обеспечивается разделением пользовательского интерфейса, бизнес-логики, навигации, хранения данных и работы с системными службами. Функциональные области оформляются как отдельные модули, доступ к репозиториям и службам выполняется через протоколы, а зависимости передаются компонентам извне. Благодаря этому изменение способа хранения не должно требовать полной переработки экранов. Алгоритмы планирования, анализа, статистики, уровней и достижений должны проверяться отдельно от интерфейса, а сервисы – допускать замену зависимостей тестовыми реализациями. Соответствие этим требованиям оценивается по структуре модулей, направлениям зависимостей и возможности воспроизводимого запуска модульных и интерфейсных тестов.

Нефункциональные требования задают ограничения, которые учитываются при проектировании и реализации Luma. Их соблюдение оценивалось при ручной проверке основных пользовательских сценариев и анализе структуры приложения.

1.4 Функциональные возможности приложения

Функциональные требования к приложению определены техническим заданием. В курсовой работе они представлены диаграммами вариантов использования: такой формат связывает возможности приложения с целями пользователя и показывает участие системных служб iOS без повторения требований отдельным перечнем.

Основным актором является пользователь. До входа он может зарегистрироваться или авторизоваться, а после входа получает доступ к планированию, дневнику, фокус-

сессиям и анализу состояния. Системные службы iOS показаны как вспомогательные акторы только в тех случаях, когда они действительно участвуют во взаимодействии.

Начало работы включает регистрацию и несколько способов входа в учётную запись (рисунок 1.1).

Рисунок 1.1 – Регистрация и авторизация

Новый пользователь создаёт учётную запись, указывая данные, предусмотренные формой регистрации. Для последующего входа доступны электронная почта и пароль. После первичной авторизации пользователь может включить вход через Face ID; системная служба выполняет биометрическую проверку, а приложение сохраняет контроль над состоянием пользовательской сессии.

После авторизации пользователь может просмотреть и изменить параметры профиля (рисунок 1.2).

Рисунок 1.2 – Профиль пользователя

В профиле отображаются сведения об учётной записи, уровень и накопленный опыт. Пользователь может изменить аватар, настроить биометрический вход или выйти из учётной записи. Служба выбора изображений предоставляет выбранный файл, а Face ID используется только для подтверждения личности.

Работа с напоминаниями и журналом уведомлений представлена на рисунке 1.3.

Рисунок 1.3 – Настройка и получение уведомлений

Пользователь самостоятельно включает или отключает уведомления и получает напоминания о запланированных действиях. Системная служба iOS отвечает за доставку уведомления. История уже доставленных сообщений хранится и просматривается внутри приложения, поэтому этот сценарий не зависит от системного центра уведомлений.

Календарь объединяет собственные задачи пользователя и внешние события, полученные из системного календаря (рисунок 1.4).

Рисунок 1.4 – Просмотр календаря и управление задачами

После выдачи разрешения приложение читает события системного календаря и учитывает занятые интервалы при составлении плана. Пользователь выбирает день, просматривает его расписание, применяет фильтр и управляет собственными задачами. Импортированные события доступны только для чтения и не редактируются в приложении.

Создание задач и событий связано с автоматическим подбором времени, что отражено на рисунке 1.5.

Рисунок 1.5 – Создание задач и автоматическое планирование

Событие создаётся для заранее выбранного времени. Для обычной задачи пользователь задаёт продолжительность и ограничения, после чего приложение ищет подходящий свободный интервал. Сложная задача дополнительно разбивается на подзадачи, которые могут быть распределены по нескольким доступным периодам.

Центральный сценарий автоматического планирования начинается после заполнения параметров задачи. Приложение сопоставляет её длительность с уже занятыми и свободными интервалами, выбирает допустимое время и добавляет результат в план дня. Если непрерывного интервала недостаточно, для сложной задачи используются отдельные подзадачи; созданное пользователем событие с фиксированным временем при этом не переносится.

Функции фиксации состояния и отображения прогресса собраны на рисунке 1.6.

Рисунок 1.6 – Достижения и дневник состояния

В дневнике пользователь отмечает эмоциональное состояние и сведения о сне. Эти записи используются при последующем анализе дня. Раздел достижений показывает текущий уровень, накопленный опыт и условия уже открытых достижений. Два набора сценариев логически независимы, но связаны с одной учётной записью.

Фокус-сессия помогает выполнять выбранную задачу без постоянного возврата в основной интерфейс приложения (рисунок 1.7).

Рисунок 1.7 – Фокус-сессия и Live Activity

Пользователь запускает сессию для выбранной задачи и наблюдает её ход в Live Activity. Через системное представление можно завершить задачу, после чего действие передаётся приложению. Служба Live Activities отображает текущее состояние и принимает команду, но инициатором сценария и получателем результата остаётся пользователь.

Итоговый анализ дня связывает данные календаря с выполненными задачами и записями о состоянии пользователя (рисунок 1.8).

Рисунок 1.8 – Анализ состояния пользователя

Приложение рассчитывает показатели нагрузки, сопоставляет их с продолжительностью сна и эмоциональными отметками, а затем формирует рекомендации. Системный календарь передаёт внешние события, необходимые для оценки занятого и свободного времени. Пользователь получает результат в виде сводки и может перейти к следующей фокус-сессии или скорректировать план.

Таким образом, диаграммы описывают путь от первого входа до ежедневного планирования, выполнения задач и оценки результатов. Далее эти сценарии используются для выделения сущностей предметной области.

1.4.1 Текстовые сценарии ключевых вариантов использования

Регистрация и вход. Предусловие: пользователь не авторизован. После открытия формы он вводит регистрационные данные либо электронную почту и пароль. Приложение проверяет обязательные поля, формат значений и наличие учётной записи. При корректных данных создаётся или восстанавливается сессия; при ошибке форма сохраняется, а пользователь получает сообщение о причине отказа. После первого входа можно включить Face ID, но неуспешная биометрическая проверка не исключает вход по паролю.

Создание события. Предусловие: пользователь авторизован и открыл форму добавления элемента. Он указывает название и фиксированный интервал времени, при необходимости добавляет описание, важность и сложность. Приложение проверяет корректность интервала и сохраняет событие без автоматического переноса. Если обязательные данные отсутствуют или окончание указано раньше начала, сохранение не выполняется.

Создание обычной задачи. Пользователь задаёт название, период выполнения, предполагаемую продолжительность, важность и сложность. Приложение получает занятые интервалы, формирует свободные промежутки и выбирает подходящее время без пересечений. Если выполнение выходит за 23:00, пользователь решает, оставить исходный интервал или перенести остаток на следующий день. Результатом становится задача с рассчитанным временем выполнения.

Создание сложной задачи. Пользователь задаёт общий период и добавляет не менее одной подзадачи с собственным названием, длительностью, важностью и сложностью. Приложение проверяет параметры и последовательно распределяет подзадачи по свободным интервалам. Если допустимого времени недостаточно, сохранение не завершается без сообщения пользователю. При успехе сохраняются родительская задача и связанные с ней подзадачи.

Учёт системного календаря. При первом обращении к календарным данным приложение запрашивает разрешение iOS. После согласия события выбранного периода загружаются как интервалы только для чтения и учитываются при отображении расписания

и подборе времени задач. При отказе или отзыве разрешения приложение продолжает работать с локальными задачами и сообщает, что доступ можно изменить в настройках iOS.

Заполнение дневника. Авторизованный пользователь выбирает запись сна или эмоционального состояния. Для сна указываются время засыпания и пробуждения, после чего длительность рассчитывается автоматически, включая переход через полночь. Для эмоции выбираются вид и интенсивность от 1 до 10, а причина может быть пропущена. После сохранения запись связывается с текущим пользователем и календарным днём.

Проведение фокус-сессии. Пользователь выбирает незавершённую задачу текущего дня, при желании включает локальный плейлист и запускает таймер. Приложение отображает оставшееся время и, если устройство поддерживает функцию, создаёт Live Activity. Завершение останавливает таймер и музыку, закрывает Live Activity, сохраняет фактическую длительность и отмечает задачу выполненной. Ошибка системной активности не должна останавливать обычный таймер.

Анализ дня. Пользователь открывает раздел анализа после добавления задач и, при наличии, дневниковых записей. Приложение получает задачи, события системного календаря, сведения о сне и эмоциях, рассчитывает индикативные показатели ресурса, нагрузки, баланса и полноты данных, затем формирует рекомендации. Если данных о состоянии недостаточно, результат помечается как предварительный. Анализ носит справочный характер и не используется для медицинской диагностики.

1.5 Модель предметной области

На основе рассмотренных сценариев была построена модель предметной области, представленная на рисунке 1.9. Она фиксирует основные понятия приложения и связи между ними, не повторяя структуру конкретных экранов или программных классов.

Рисунок 1.9 – Модель предметной области

Центральной сущностью является пользователь. С ним связаны созданные задачи, записи сна и эмоций, фокус-сессии и достижения. Один пользователь может иметь несколько объектов каждого типа, при этом каждая запись относится к конкретной учётной записи.

В предметной модели задача и событие рассматриваются как разные виды планируемого элемента. Задача имеет состояние выполнения, предполагаемую продолжительность и может переноситься или разделяться на подзадачи. Событие занимает фиксированный временной интервал и не участвует в автоматическом переносе; импортированное событие дополнительно хранит источник и доступно только для чтения.

Подзадача связана с родительской задачей, а фокус-сессия – с тем действием, для которого пользователь проводил сеанс концентрации.

Для фокус-сессии может быть выбран плейлист, содержащий один или несколько музыкальных треков. Музыка является дополнительной частью сценария и не влияет на возможность провести фокус-сессию без звукового сопровождения.

Записи сна и эмоций принадлежат пользователю и используются при построении статистики и анализа состояния. Достижения также связаны с учётной записью и отражают прогресс, полученный за выполнение задач, ведение дневника и проведение фокус-сессий.

По итогам анализа определены границы Luma, ключевые пользовательские сценарии, требования к качеству и состав предметной модели. Эти результаты используются в следующей главе для выбора платформенных средств, архитектуры и способов хранения данных.

2 Проектирование

Проектные решения для Luma опираются на требования к платформе, интерфейсу, локальному хранению, системным интеграциям и автономной работе. Отдельно учтено разделение ответственности между компонентами.

2.1 Выбор платформы и средств разработки

Технические данные определяют Luma как нативное приложение для iPhone. Поэтому выбор платформы не требовал сравнения с Android или кроссплатформенными решениями. Основные проектные решения касались UI-фреймворка, среды разработки, языка, хранения данных и системных API.

2.1.1 Выбор UI-фреймворка

Основная часть интерфейса реализована на UIKit. Фреймворк предоставляет оконную и событийную инфраструктуру iOS, базовые представления, элементы управления и контроллеры навигации [10]. В Luma на UIKit построены формы авторизации и создания задач, календарь, дневник, статистика, профиль и координаторы переходов.

UIKit выбран основным UI-фреймворком благодаря императивной модели управления интерфейсом. Она позволяет явно контролировать создание и конфигурацию UIView, жизненный цикл UIViewController, обработку событий, иерархию представлений и навигационные переходы. Такой уровень управления важен для форм с валидацией, календаря со сложной компоновкой, многошагового создания задач и координаторной навигации Luma. Программная вёрстка в UIKit также даёт возможность точно задавать ограничения Auto Layout, порядок обновления элементов и реакцию экрана на действия пользователя. Поэтому UIKit используется как единая основа основной части интерфейса.

SwiftUI выбран для экрана достижений и интерфейса Live Activity, поскольку эти части преимущественно отображают состояние и требуют автоматического обновления представления при изменении данных. Декларативная модель SwiftUI позволяет описывать соответствие между состоянием и интерфейсом без ручного управления каждым обновлением элементов [11]. Для экрана достижений это упрощает отображение списка, прогресса и состояний блокировки, а для Live Activity соответствует модели представления ActivityKit и WidgetKit на экране блокировки и в Dynamic Island [12; 13]. Экран достижений встраивается в основной UIKit-интерфейс через UIHostingController. UIKit используется для сценариев со сложным управлением экраном и навигацией, а SwiftUI – для изолированных представлений, содержание которых определяется текущим состоянием данных.

На развёрнутой карточке достижения Core Motion передаёт данные о наклоне устройства и используется для трёхмерного поворота карточки [14]. Эффект отключается при системной настройке уменьшения движения. Core Motion выбран вместо самостоятельной обработки низкоуровневых данных датчиков, поскольку входит в SDK iOS и предоставляет готовые обработанные значения ориентации.

2.1.2 Выбор инструментов разработки

Основной средой разработки выбран Xcode. Она объединяет редактор исходного кода, систему сборки, Simulator, отладчик, средства тестирования и Instruments [15]. Код написан на Swift; проект настроен на Swift 5. Официальная спецификация языка используется как базовый источник по синтаксису и типам [16]. Xcode и Swift выбраны из-за прямой поддержки iOS SDK и возможности собирать основное приложение вместе с расширением Live Activity в одном проекте. Git используется для фиксации изменений и возврата к предыдущим состояниям проекта [17].

Для локального хранения выбран Core Data: он описывает сущности и связи объектной модели и сохраняет данные пользователя на устройстве [18]. Через него в проекте хранятся задачи, дневниковые записи, фокус-сессии и достижения. Небольшие секретные значения отделены от основной базы: Security предоставляет Keychain Services для защищённого хранения [19], а CryptoKit рассчитывает SHA-256 для проверки пароля и формирования устойчивых идентификаторов импортированных событий [20]. Это разделение не помещает данные аутентификации в общую модель Core Data.

EventKit выбран для чтения событий системного календаря [21], UserNotifications – для локальных напоминаний и получения списка доставленных уведомлений [22], LocalAuthentication – для входа через Face ID [23]. ActivityKit управляет жизненным циклом Live Activity [12], а AVFoundation воспроизводит локальные аудиозаписи во время фокус-сессии [24]. Эти средства входят в системные SDK, поэтому используют штатные механизмы разрешений и жизненного цикла iOS без подключения сторонних библиотек.

Использование системных средств уменьшает число внешних зависимостей и обеспечивает согласованную работу с механизмами iOS. Перед обращением к защищённым данным приложение использует штатные запросы разрешений. Конкретная минимальная версия системы задаётся конфигурацией сборки и должна соответствовать доступности используемых API.

2.2 Архитектурный подход

Для приложения выбран MVVM-C с сервисным и репозиторным слоями. Подход разделяет отображение, состояние экрана, навигацию, прикладные операции и хранение данных. Общая схема взаимодействия компонентов представлена на рисунке 2.1.

Рисунок 2.1 – Архитектурный подход, используемый в приложении

2.2.1 Распределение ответственности

На уровне представления View и ViewController отвечают за отображение данных, обработку событий интерфейса и передачу действий пользователя в ViewModel. ViewModel хранит состояние экрана, подготавливает данные для отображения и вызывает прикладные сервисы, но не управляет переходами между экранами. Coordinator создаёт экраны, передаёт им зависимости и определяет последовательность переходов. Поэтому изменение маршрута не требует переносить код навигации между контроллерами [25–27].

В сервисном слое сосредоточены операции с учётной записью, задачами, дневником, анализом и достижениями. RepositoryProtocol задаёт интерфейс доступа к данным, а Repository реализует этот интерфейс для Core Data. Mapper преобразует Storage Model в Domain Model и обратно, поэтому доменные структуры не обязаны знать о формате хранения. Storage и Storage Model отвечают за сохранение и извлечение объектов. Сервисный слой зависит от абстракции репозитория, а не от конкретного способа хранения, что соответствует принципу разделения слоёв [28].

2.2.2 Обоснование выбора MVVM-C

Классический MVC является штатным и понятным для UIKit подходом: Controller связывает Model и View и реагирует на действия пользователя [29]. Для небольшого числа экранов это сокращает число типов. В Luma один контроллер при таком подходе должен был бы одновременно вести состояние экрана, обрабатывать формы, открывать следующие экраны и обращаться к сервисам. По мере роста функциональных разделов это увеличило бы объём контроллеров и затруднило бы их изолированную проверку.

MVP также выносит логику из представления: пассивная View передаёт события Presenter, а Presenter вызывает методы интерфейса View. Преимуществом стала бы простая подмена View в тестах. Однако для каждого экрана потребовались бы отдельный протокол и ручные команды обновления состояния. Сам MVP не задаёт отдельной границы для навигации, поэтому проблему многошаговых сценариев всё равно пришлось бы решать дополнительно. Паттерн Passive View и его издержки при ручной синхронизации описаны в [30].

MVVM-C выбран потому, что он разделяет две наиболее изменчивые части: ViewModel описывает состояние и операции экрана, а Coordinator – переходы между экранами. Это соответствует структуре Luma, где есть несколько независимых разделов, многошаговое создание задачи, общие данные для календаря, статистики и анализа, а также одновременно используются UIKit и SwiftUI. ViewModel можно проверять без запуска всего интерфейса, а Coordinator позволяет менять маршрут, не затрагивая логику экрана.

2.2.3 Преимущества выбранного подхода

К преимуществам выбранного подхода относятся явное распределение ответственности, сокращение кода в ViewController, повторное использование сервисов на разных экранах и возможность подмены части зависимостей в тестах. AppCoordinator выбирает стартовый поток, AuthCoordinator управляет авторизацией, TabBarCoordinator создаёт основные разделы, а AddTaskCoordinator ведёт пользователя от выбора типа задачи до результата планирования. DIContainer создаёт общие экземпляры сервисов и передаёт их координаторам и ViewModel.

2.3 Хранение данных и системные интеграции

Требования к автономной работе, целостности и конфиденциальности предполагают локальное хранение пользовательских данных, связь записей с владельцем и отсутствие обязательной передачи данных внешним сервисам. Для этого выбран NSPersistentContainer и репозитории Core Data. В хранилище представлены UserEntity, TaskEntity, SleepEntryEntity, EmotionEntryEntity, FocusSessionEntity и AchievementEntity. Задачи, записи дневника, фокус-сессии и достижения связываются с UserEntity отношением owner; подзадачи представлены тем же типом TaskEntity и связаны с родительской задачей отношением parentTask. Репозитории фильтруют выборки по идентификатору владельца, а TaskRepository отдельно обрабатывает создание и удаление подзадач. Модель локального хранилища представлена на рисунке 2.2.

Рисунок 2.2 – Модель локального хранилища Core Data

Выбор Core Data связан с необходимостью хранить взаимосвязанные локальные объекты и получать выборки по пользователю и временному интервалу. PersistenceController поддерживает автоматическую лёгкую миграцию и откатывает контекст при ошибке сохранения. Репозитории централизуют операции с данными, поэтому ViewModel не обращаются к управляемому контексту напрямую.

Для учётных данных выбран отдельный путь хранения. UserRepository сохраняет профиль в Core Data, а KeychainManager вычисляет SHA-256 от пароля и строки

идентификатора пользователя и помещает результат в Keychain. LocalAuthentication применяется для проверки Face ID системными средствами. Такое разделение не допускает хранения пароля в открытом виде и использует штатный биометрический механизм iOS.

Системный календарь подключён через CalendarServiceProtocol и CalendarEventService. Сервис запрашивает доступ к EventKit, преобразует полученные события в TaskItem только для чтения и при отсутствии разрешения возвращает пустой список, поэтому внутренние задачи могут обрабатываться без календарных событий. Напоминания создаёт LocalNotificationService; музыка воспроизводится FocusMusicPlayer; Live Activity запускает и завершает FocusLiveActivityManager; изображение профиля сохраняет AvatarStore, предварительно уменьшая его до 400 × 400 точек в фоновой очереди. Эти компоненты выбраны для реализации требований к календарю, уведомлениям, фокус-сессии, локальным ресурсам и обработке изображения.

Данные и музыкальные ресурсы обрабатываются локально, а обязательные сетевые клиенты в основных сценариях не используются. Благодаря этому управление задачами, дневник, фокус-сессии, статистика и основной процесс планирования не требуют постоянного сетевого соединения.

2.4 Проектирование пользовательского интерфейса

Требования к интерфейсу задают единый визуальный стиль, постоянный доступ к основным разделам, понятные формы, сообщения об отсутствии данных и ошибках, подтверждение значимых действий и русский язык. Для реализации выбран единый контейнер MainTabBarController с пользовательской панелью CustomTabBar. TabBarCoordinator формирует пять основных разделов: календарь, дневник, общий анализ, подробную статистику и профиль. Переход к ним выполняется выбором соответствующего элемента панели, а вложенные сценарии открываются координаторами функциональных разделов.

Повторяющиеся элементы вынесены в компоненты CardContainerView, CustomButton, CustomTextField, FormFieldCard, DatePickerCard, MetricSliderCard, ValidationHintView и EmptyStateView. Цвета и часть визуальных параметров собраны в расширениях DesignTokens. Формы создания задач используют TaskValidator и SubtaskValidator, а формы авторизации – LoginValidator и RegistrationValidator. Auto Layout и привязка ряда экранов к safeAreaLayoutGuide выбраны для адаптации компоновки и исключения наложения на системные области. Примеры повторно используемых элементов интерфейса приведены на рисунке 2.3.

Рисунок 2.3 – Повторно используемые элементы пользовательского интерфейса

Повторное использование компонентов и общих визуальных параметров поддерживает единый стиль экранов. Для адаптации компоновки применяются Auto Layout и safeAreaLayoutGuide. На элементах экрана анализа также заданы accessibility-метки и поддержка Dynamic Type.

В результате проектирования выбраны нативные средства iOS, сочетание UIKit и SwiftUI, архитектура MVVM-C, локальное хранение Core Data и отдельные сервисы системных интеграций. Такое разделение соответствует сценариям Luma и позволяет реализовать основную логику независимо от конкретных экранов.

3 Реализация приложения

В главе рассматривается реализация Luma через основные пользовательские сценарии. Для каждого сценария описаны входные данные, ключевой механизм обработки, результат и поведение при недоступности системной функции или некорректном вводе.

Общий поток обработки начинается с действия пользователя в UIKit- или SwiftUI-интерфейсе. Модель представления проверяет состояние экрана и передаёт данные прикладному сервису; сервис выполняет операцию через репозиторий либо системный API, а полученный результат возвращается в интерфейс. Координаторы отвечают за переходы, а DIContainer передаёт компонентам необходимые зависимости.

3.1 Учётная запись, профиль и сессия

Сценарий учётной записи должен обеспечить регистрацию, обычный и биометрический вход, восстановление сессии и безопасный выход. Пользователь вводит имя, дату рождения, адрес электронной почты и пароль; приложение проверяет обязательные значения и создаёт локальную учётную запись. После входа профиль предоставляет доступ к данным пользователя, аватару, уровню, опыту и системным настройкам. На устройстве без Face ID биометрическая настройка скрывается, а ошибка проверки не блокирует вход по паролю. Основные экраны сценария показаны на рисунке 3.1.

Рисунок 3.1 – Экраны входа через Face ID, авторизации и регистрации

Формы регистрации и входа реализованы отдельными модулями RegistrationViewController/ViewModel и LoginViewController/ViewModel. Проверка значений вынесена в RegistrationValidator и LoginValidator: до обращения к хранилищу проверяются обязательность полей, формат введённых данных и длина пароля не менее шести символов. После успешной проверки AuthService вызывает UserRepository. При регистрации репозиторий создаёт локального пользователя, а при входе ищет запись по нормализованному адресу электронной почты и проверяет учётные данные. Сценарий включения биометрического входа представлен на рисунке 3.2.

Рисунок 3.2 – Включение входа через Face ID

Пароль не записывается в модель Core Data. В текущей реализации KeychainManager вычисляет SHA-256-хеш из пароля и индивидуальной соли на основе UUID пользователя,

а результат сохраняет в Keychain. Это исключает хранение пароля в открытом виде, однако SHA-256 является быстрой хеш-функцией общего назначения, а не специализированной функцией выработки ключа для паролей. Поэтому данное решение рассматривается как ограничение локального прототипа; перед эксплуатационным выпуском механизм следует заменить на PBKDF2, scrypt или Argon2 с документированными параметрами.

Состояние входа хранит SessionManager. При запуске AppCoordinator выбирает один из трёх маршрутов: восстановление основной сессии без повторного ввода, попытка биометрической разблокировки или показ экрана авторизации. Включение Face ID выполняет ProfileViewModel: сначала проверяется наличие именно Face ID через LocalAuthentication, затем запускается системная биометрическая проверка, и только после успеха настройка связывается с текущим идентификатором пользователя. Если технология недоступна или проверка неуспешна, настройка остаётся выключенной, а обычная форма входа остаётся доступной. Последовательность авторизации с помощью Face ID представлена на рисунке А.1 приложения А.

Такое разделение важно для границ сессии. Сохранённый идентификатор пользователя сам по себе не заменяет биометрическую проверку, если для учётной записи включён Face ID: координатор сначала открывает отдельный экран разблокировки. При отключённой биометрии сохранённая сессия может быть восстановлена напрямую. После выхода SessionManager удаляет состояние активного пользователя, поэтому повторный доступ к данным проходит через авторизацию. В коде также проверяется, что сохранённый для Face ID идентификатор относится к текущей учётной записи; это предотвращает использование настройки другой локальной записи.

Профиль отображает имя, адрес электронной почты, маску пароля, состояние системных настроек, аватар и рассчитанный прогресс уровня. Изменение аватара выполняется через системный выбор изображения и AvatarStore; перед сохранением изображение уменьшается. На экране также размещены настройки Face ID и уведомлений. Выход выполняется после подтверждения, очищает активную сессию, возвращает пользователя на экран авторизации и отменяет локальные уведомления.

При корректном вводе пользователь получает активную сессию и доступ к собственным данным. Ошибки формата, неизвестная учётная запись и неуспешная биометрическая проверка возвращаются на уровень формы без изменения сохранённых данных. После выхода сессия очищается, уведомления отменяются, а следующий доступ требует повторной авторизации.

3.2 Календарь и управление элементами

Календарь должен давать пользователю единое представление локальных задач и занятости из системного календаря. После выбора даты элементы сортируются по времени, фильтруются по состоянию выполнения и участвуют в расчёте дневного прогресса. Собственные задачи можно изменять, завершать и удалять; события iOS остаются доступными только для чтения. Последовательность взаимодействия с системным календарём приведена на рисунке А.2 приложения А.

TabBarCoordinator создаёт пять разделов: календарь, дневник, анализ дня, статистику и профиль. Экран календаря передаёт выбранную дату и фильтр в CalendarViewModel. Модель представления получает данные через TaskService, после чего передаёт представлению отфильтрованный список и пару значений для прогресса. При расчёте сложная задача раскрывается до подзадач: завершённые подзадачи участвуют в числителе, все подзадачи – в знаменателе. Этот же принцип применяется при фильтрации, поэтому сложная задача не скрывает состояние отдельных этапов.

Собственные элементы загружаются из TaskRepository, а события iOS – через CalendarEventService, использующий EventKit. CalendarEventTranslator преобразует внешнее событие в общую доменную модель TaskItem и помечает источник как календарный. TaskService объединяет оба массива и сортирует их по фактическому времени начала. Преобразование не переносит внешнюю запись в Core Data и не предоставляет методов записи в EventKit. В интерфейсе проверяется источник: редактирование системного события не открывается, тогда как для внутренней задачи доступны изменение названия, описания, важности и сложности, переключение выполнения и удаление.

Общая доменная модель упрощает визуальное объединение, но источник сохраняется как обязательное различие поведения. Внутренняя задача имеет владельца, состояние выполнения и операции репозитория. Внешнее событие выступает временным ограничением: его границы используются при отображении и автоматическом планировании, но жизненным циклом управляет системный календарь. Благодаря этому отказ в разрешении EventKit уменьшает только состав объединённого списка и не меняет сохранённые в Core Data задачи пользователя.

Обновление организовано событиями. CalendarViewModel наблюдает собственное уведомление об изменении хранилища задач, EKEventStoreChanged и UIApplication.didBecomeActiveNotification. Серия близких событий сворачивается задержкой 0,2 секунды, после чего список перечитывается. При отказе в доступе к системному календарю сервис возвращает пустой набор внешних событий, поэтому внутренний календарь продолжает работать.

Экран календаря, на котором отображаются выбранная дата, дневной прогресс, фильтр и состояние задач, приведён на рисунке 3.3.

Навигация, выбор даты, фильтры, разворачивание подзадач, расчёт прогресса и операции над собственными задачами объединены в экране календаря. События iOS добавляются в общий список в режиме чтения, тогда как внутренние задачи можно изменять, завершать и удалять. CalendarViewModel реагирует на изменения EventKit и возврат приложения в активное состояние.

Рисунок 3.3 – Календарь и отображение задач в процессе

3.3 Автоматическое планирование и граница 23:00

При добавлении элемента пользователь выбирает событие, обычную или сложную задачу. Событие сохраняется в указанном интервале. Для обычной задачи приложение подбирает свободное время по длительности, важности и сложности, а сложную задачу предварительно раскладывает на подзадачи. Планирование учитывает локальные задачи и события системного календаря, исключает пересечения и преимущественно использует диапазон 08:00–23:00. Перенос остатка после 23:00 выполняется только с согласия пользователя. Алгоритм распределения обычной задачи представлен на рисунке 3.4.

Рисунок 3.4 – Алгоритм распределения обычной задачи

Выбор вида элемента хранит AddTaskFlowViewModel, а формы собраны в модуле AddTask. Для события используются введённые пользователем начало и конец: оно сохраняется без автоматического переноса и затем рассматривается как занятый интервал. Для обычной и сложной задач TaskService сначала получает внутренние элементы за период, затем добавляет считанные из EventKit интервалы и передаёт общий набор алгоритмам из TaskScheduling.swift. Введённая длительность нормализуется минимум до 15 минут при вычислении плана.

TaskAutoScheduler строит рабочие интервалы каждого дня с 08:00 до 23:00, вычитает пересекающиеся занятые отрезки и формирует допустимые кандидаты. Оценка кандидата учитывает день периода, загруженность, важность, сложность и более раннее размещение. Выбирается кандидат с максимальной оценкой. Если внутри рабочих сегментов подходящего окна нет, применяется nextNonOverlappingStart, который сдвигает начало за пересекающиеся интервалы.

Коэффициенты образуют детерминированную эвристику Luma: при одинаковых входных данных она выбирает одинаковый вариант и отдаёт дополнительный вес более раннему времени для важных и сложных дел. Требование «по возможности раньше» тем самым преобразуется в предпочтение при выборе свободного интервала. Кандидаты основного пути формируются внутри свободных промежутков, что исключает пересечения с занятыми интервалами.

Для сложной задачи SubtaskAutoScheduler сначала вычисляет срочность каждой подзадачи из важности и сложности, задаёт ориентир внутри общего периода и последовательно подбирает свободные интервалы. После добавления очередного плана этот интервал становится занятым для следующих подзадач. Родительская задача сохраняется с границами от начала первой до конца последней подзадачи, а каждая подзадача получает parentTaskId. До подтверждения формы черновики подзадач существуют только в памяти интерфейса; после сохранения это отдельные сущности Core Data со структурной связью с родителем.

Алгоритм на рисунке 3.4 отражает базовый механизм исключения пересечений: курсор последовательно перемещается к концу занятого интервала, а кандидаты создаются только в образовавшихся разрывах. Коэффициенты авторской эвристики поясняются в тексте, чтобы не перегружать диаграмму деталями реализации. Механизм переноса части задачи после 23:00 показан на рисунке 3.5.

Рисунок 3.5 – Механизм переноса части задачи после 23:00

После создания обычной задачи AddTaskCoordinator проверяет результат через lateBoundary. Если начало находится до 23:00, а окончание – после этой границы, появляется диалог с вариантами «Оставить как есть» и «Разделить». До выбора положительного действия разделение не выполняется. При согласии первая запись сокращается до 23:00, а остаток создаётся на следующий день с 08:00 и названием с пометкой «продолжение». При отказе сохраняется исходный вариант. Результат выбора пользователя показан на рисунке 3.6.

Рисунок 3.6 – Перенос части задачи после 23:00

3.4 Уведомления и дневник

Уведомления помогают не пропустить начало задачи и заполнение дневника, но используются только после разрешения пользователя. Для задач создаются напоминания за 30 и 5 минут, а для дневников – в заданные моменты дня; после сохранения записи оставшиеся напоминания этого типа отменяются. В дневнике эмоций сохраняются вид,

интенсивность 1–10 и необязательная причина, а в дневнике сна – время засыпания и пробуждения с расчётом длительности, включая переход через полночь. Доставленные уведомления доступны в отдельном списке.

ProfileViewModel передаёт изменение переключателя в LocalNotificationService. При первом включении сервис запрашивает у UNUserNotificationCenter разрешения alert, sound и badge. Если система возвращает отказ, переключатель сбрасывается, а модель представления выдаёт сообщение о необходимости изменить настройки iOS. При выключении очищаются все ожидающие и все уже доставленные уведомления приложения. Аналогичная полная отмена выполняется при выходе из учётной записи.

Для простой задачи создаются идентификаторы напоминаний на 30 и 5 минут до начала. Сложная задача предварительно раскрывается до подзадач. Если момент тридцатиминутного напоминания уже прошёл, оно не добавляется. Если до старта остаётся менее пяти минут, сервис создаёт ближайшее актуальное напоминание.

Идентификатор напоминания включает UUID задачи и контрольный интервал. Перед повторным планированием запрос с тем же идентификатором удаляется. Это предотвращает дублирование одного и того же напоминания.

Дневниковое расписание строится на семь ближайших дней. Перед добавлением уведомлений сервис проверяет наличие записи соответствующего типа. После сохранения DiaryViewModel отменяет оставшиеся напоминания эмоций или сна на текущий день. Экран истории получает доставленные уведомления от системного центра и сортирует их по убыванию даты.

Эмоциональный дневник предоставляет одиночный выбор: EmotionGridView хранит один selectedIndex, а обработчик сохранения формирует один элемент ввода. Перечень в доменной модели точно совпадает с техническим заданием:

Проверка модели показала, что за одно сохранение пользователь выбирает одну эмоцию из шести: радость, грусть, злость, тревогу, покой или любовь. Интенсивность при сохранении ограничивается диапазоном 1–10, причина может отсутствовать. DiaryRepository связывает запись с пользователем и календарным днём. Для сна время относится к выбранному дню, а пробуждение переносится на следующие сутки, если оно не позже времени засыпания; после этого длительность вычисляется в минутах.

Типы дневниковых записей, перечень эмоций, одиночный выбор, интенсивность, необязательная причина, переход сна через полночь и отмена дневниковых напоминаний объединены в DiaryViewModel, DiaryService и DiaryRepository. Настройка уведомлений, их история и общая отмена выполняются LocalNotificationService.

3.5 Фокус-сессия и Live Activity

Фокус-сессия связывает обратный отсчёт с выбранной незавершённой задачей текущего дня. Пользователь может запустить сессию без музыки или выбрать локальный плейлист. Во время работы приложение обновляет таймер раз в секунду и, если функция поддерживается, передаёт состояние в Live Activity. Завершение останавливает таймер и музыку, сохраняет фактическую длительность и отмечает задачу выполненной; ошибка ActivityKit не должна прерывать основной сценарий.

FocusViewModel получает элементы текущего дня через тот же TaskService, что и календарь. Сложные задачи раскрываются до незавершённых подзадач. Для выбранного объекта создаётся FocusTaskItem, а продолжительность определяется по estimatedDuration; если она отсутствует, используется разница между концом и началом, с нижней границей одна минута. Локальные плейлисты перечислены в FocusMusicLibrary, воспроизводятся FocusMusicPlayer, а режим без музыки не вызывает проигрыватель.

После старта FocusViewController сохраняет время начала и окончания, запускает Timer с периодом одна секунда и просит FocusLiveActivityManager создать системную активность. Если ActivityKit недоступен или запрос завершился ошибкой, обычный таймер продолжает работу. При завершении контроллер инвалидирует таймер, скрывает карточку отсчёта, завершает Live Activity, останавливает музыку, сохраняет фактический интервал фокус-сессии и отмечает локальную задачу выполненной.

Сохранённая фокус-сессия отделена от плановой длительности задачи. В запись попадают фактические моменты начала и окончания, а репозиторий достижений использует их разницу при пересчёте накопленного времени. Поэтому досрочное завершение не приписывает пользователю всю запланированную продолжительность. Одновременно задача завершается только один раз: при возврате из Live Activity экран получает уведомление и завершает локальное состояние сеанса без повторного изменения задачи.

Расширение Live Activity реализовано на SwiftUI. На экране блокировки оно показывает название и тип задачи, таймер и действие завершения; для Dynamic Island определены развёрнутое, компактное и минимальное состояния. Действие формирует внутреннюю ссылку с идентификатором задачи. AppCoordinator проверяет активную сессию, завершает задачу через TaskService, закрывает активность и сообщает экрану фокуса, чтобы тот не повторял операцию завершения.

Согласованность состояния проверяется по совпадению названия, типа задачи и оставшегося времени между приложением и системным интерфейсом; действие «Завершить задачу» остаётся доступным в Live Activity.

Таймер, музыка, запись сессии, код расширения и обработка внутренней ссылки объединены в общем сценарии фокус-сессии. FocusViewController управляет обратным отсчётом и воспроизведением, FocusLiveActivityManager синхронизирует системную активность, а TaskService фиксирует завершение выбранной задачи.

3.6 Анализ дня, статистика и достижения

Анализ дня связывает план пользователя с доступными сведениями о его состоянии. При расчёте учитываются задачи текущего и следующего дня, сон, эмоции, просроченные элементы, уже выполненная работа и интервалы системного календаря. Модель является авторской эвристикой: она формирует ориентировочную оценку и рекомендации, не является медицинским инструментом и не подтверждает причинную связь между состоянием и результативностью. Подготовка показателей сна и эмоционального состояния показана на рисунке 3.8, последующие этапы – на рисунках Б.1–Б.4 приложения Б.

В расчёте используются четыре нормированных показателя. Ресурс – индикативная оценка доступного состояния по шкале от 0 до 100, полученная из записей сна и эмоций. Нагрузка – оценка объёма и трудоёмкости запланированной деятельности по шкале от 0 до 100 с учётом длительности, сложности, выполненной работы, просрочек и свободного времени. Баланс – приведённое к шкале 0–100 соотношение ресурса и нагрузки; значение 50 соответствует их условному равновесию. Достоверность – показатель полноты исходных данных по шкале от 0 до 100, а не статистическая точность или медицинская валидность результата.

Рисунок 3.8 – Подготовка показателей сна и эмоционального состояния

Сбор исходных данных выполняет AnalysisService. Сервис получает сведения о пользователе, записи дневников, задачи текущего и следующего дня, а также календарные события. Сложные задачи раскрываются до отдельных подзадач, чтобы расчёт отражал фактический объём работы. События системного календаря рассматриваются как фиксированные интервалы: они уменьшают доступное время, но не включаются в объём гибких задач, которыми пользователь может управлять внутри приложения.

Ресурс рассчитывается по доступным сведениям о сне и эмоциональном состоянии. Оценка сна учитывает последнюю запись и среднее значение за семь дней, а рекомендуемая продолжительность выбирается по возрастной группе. В авторской эвристике сну присвоен вес 0,65, поскольку он представлен количественной продолжительностью, а эмоциональным данным – 0,35. Эти коэффициенты не являются клинически

валидированными и рассматриваются как настраиваемые параметры прототипа. Если обе группы сведений отсутствуют, используется нейтральное значение 50.

Нагрузка учитывает плановую продолжительность, сложность и важность задач, уже затраченное усилие, просрочки и фрагментацию списка. Интервалы системного календаря уменьшают доступное время. Итог ограничивается диапазоном 0–100. Баланс вычисляется как 50 + 0,65 × (ресурс − нагрузка) и также ограничивается шкалой 0–100; поэтому одинаковое число задач может давать разный результат при различной трудоёмкости и занятости дня.

Достоверность зависит только от полноты исходных данных. Начальное значение повышается при наличии даты рождения, записи сна за текущий день, не менее трёх записей сна за неделю и актуальной эмоциональной записи. Значения ниже 45 считаются низкими, от 45 до 74 – средними, от 75 – высокими. При низкой достоверности приложение показывает предварительный анализ и не подменяет отсутствующие наблюдения категоричным выводом.

На экране анализа пользователь видит итоговые оценки ресурса, нагрузки и баланса, уровень уверенности и сформированные рекомендации. Это позволяет не только получить числовой результат, но и понять, какие данные повлияли на него и какое действие можно выполнить далее. Пример отображения анализа дня и рекомендаций приведён на рисунке 3.9.

Раздел статистики дополняет анализ дня сведениями за более продолжительный период. Недельная сводка содержит количество выполненных и общее количество задач, показатель качества сна, эмоциональный показатель и эффективность выполнения плана. Для расчёта используются записи от начала дня шесть суток назад до начала следующих суток. Эффективность выражает долю взвешенного объёма выполненных задач с учётом их длительности, сложности и важности, а эмоциональный показатель переводит средний баланс эмоций в нормированный диапазон от 0 до 1.

Подробные представления позволяют рассмотреть распределение эмоций и задач за 30 дней, изменение продолжительности сна, а также состав завершённых и незавершённых задач выбранного дня. После изменения задачи, сохранения дневниковой записи или завершения фокус-сессии модели представления повторно запрашивают данные. Благодаря этому общая сводка и детальные экраны используют одно актуальное состояние хранилищ. Экраны статистики эмоций, задач и сна приведены на рисунке 3.10.

Система достижений связывает регулярные действия пользователя с долгосрочным прогрессом. При первом обращении AchievementRepository создаёт каталог достижений, а затем проверяет условия по нескольким группам показателей: количеству завершённых задач, числу уникальных дней с дневниковыми записями, продолжительности сна, дням с

выраженными эмоциональными оценками и накопленному времени фокус-сессий. Пока условие не выполнено, достижение остаётся заблокированным. После выполнения оно становится разблокированным и доступным для открытия пользователем, а после открытия переходит в окончательное состояние.

Открытие достижения начисляет 50 единиц опыта, а каждые 250 единиц повышают уровень пользователя. Эти значения являются конфигурируемыми параметрами прототипа и не рассматриваются как доказанный способ повышения мотивации; их можно корректировать после анализа пользовательской обратной связи. На основании уровня выбирается звание и рассчитывается прогресс до следующего уровня, который отображается в достижениях и профиле.

Последовательность загрузки сведений о задачах, дневниковых записях и фокус-сессиях, расчёта статистических показателей, пересчёта достижений и обновления пользовательского интерфейса представлена на рисунке А.3 приложения А. Диаграмма показывает, как StatisticsViewModel обращается к AnalysisService и репозиториям, а также как открытие достижения приводит к начислению опыта и обновлению уровня пользователя.

Рисунок 3.9 – Анализ дня и рекомендации

Рисунок 3.10 – Статистика эмоций, задач и сна

3.7 Пользовательское исследование восприятия интерфейса

После завершения разработки Luma проведено пользовательское исследование восприятия интерфейса. Исследование не являлось функциональным испытанием или приёмкой приложения: оно позволяло оценить субъективное впечатление респондентов от понятности, читаемости, навигации и визуального оформления, но не подтверждало корректность реализации отдельных требований технического задания.

Сбор ответов выполнялся с помощью Google Forms. В исследовании приняли участие 14 респондентов. Участники оценивали готовое приложение Luma. Период проведения, единый набор пользовательских действий, продолжительность знакомства и состав устройств отдельно не фиксировались.

Основная часть анкеты включала утверждения, оцениваемые по пятибалльной шкале: 1 соответствовала минимальной, а 5 – максимальной выраженности оцениваемого признака. Для каждого критерия рассчитаны средняя оценка, число ответов 4–5 и

распределение ответов по шкале. Отдельно респонденты оценивали интерфейс по десятибалльной шкале и отвечали на вопрос о готовности продолжить использование приложения. Сводные результаты представлены в таблице 3.1.

Таблица 3.1 – Сводные результаты пользовательского исследования

Критерий	**Средняя оценка**	**Оценки 4–5**	**Распределение ответов 1 / 2 / 3 / 4 / 5**
Понимание назначения приложения	4,29	12 из 14	0 / 1 / 1 / 5 / 7
Понятность структуры главного экрана	4,14	11 из 14	0 / 0 / 3 / 6 / 5
Поиск основных разделов	4,14	11 из 14	0 / 1 / 2 / 5 / 6
Логичность навигации	3,79	9 из 14	0 / 2 / 3 / 5 / 4
Удобство чтения текста	3,86	9 из 14	0 / 1 / 4 / 5 / 4
Визуальная привлекательность дизайна	4,36	11 из 14	0 / 0 / 3 / 3 / 8

Продолжение таблицы 3.1

Критерий	**Средняя оценка**	**Оценки 4–5**	**Распределение ответов 1 / 2 / 3 / 4 / 5**
Современность дизайна	4,14	10 из 14	0 / 0 / 4 / 4 / 6
Подбор цветов	4,14	10 из 14	0 / 2 / 2 / 2 / 8
Аккуратность отступов, карточек, кнопок и компоновки	4,07	10 из 14	0 / 2 / 2 / 3 / 7
Понятность главных действий на экране	4,14	12 из 14	0 / 1 / 1 / 7 / 5
Удобство форм, календаря, выбора дат и задач	3,86	8 из 14	0 / 1 / 5 / 3 / 5

Наиболее высокую среднюю оценку получила визуальная привлекательность дизайна – 4,36 из 5. Понимание назначения приложения оценено в среднем на 4,29; 12 из 14 участников выбрали значения 4 или 5. Эти ответы показывают, что назначение Luma и общий визуальный образ в основном воспринимались положительно. Распределение оценок понимания назначения представлено на рисунке 3.11.

Более сдержанные оценки получены для логичности навигации – 3,79, удобства чтения текста – 3,86, а также удобства форм, календаря и выбора дат и задач – 3,86. При этом для последнего критерия оценки 4–5 дали 8 из 14 участников. Такое распределение не свидетельствует о непригодности интерфейса, но указывает на направления следующей итерации: упрощение формы создания задачи, повышение заметности выбора даты, уточнение подписей элементов навигации и проверку читаемости текста. Для иллюстрации этой зоны улучшения распределение приведено на рисунке 3.12.

Средняя общая оценка интерфейса составила 8,21 из 10, медианная – 8,5. На вопрос о готовности продолжить использование 13 из 14 респондентов ответили «Да» или «Скорее да». Эти показатели дополняют оценки отдельных критериев и характеризуют общее впечатление участников, однако не позволяют судить о фактической частоте дальнейшего использования. Распределение общей оценки интерфейса приведено на рисунке 3.13.

В основном тексте приведены три диаграммы, отражающие понимание назначения приложения, удобство форм и календаря и общую оценку интерфейса. Первичная выгрузка Google Forms должна храниться вместе с материалами исследования для последующей проверки расчётов.

При интерпретации результатов необходимо учитывать ограничения исследования. Выборка включала 14 человек, поэтому оценки описывают мнение только данной группы. Способ набора участников, демографические характеристики, единый набор действий, продолжительность знакомства и состав устройств не фиксировались. До сверки с первичной выгрузкой Google Forms численные значения рассматриваются как авторская сводка. Поэтому результаты можно называть предварительной оценкой восприятия интерфейса, но не подтверждением полной готовности продукта.

Рисунок 3.11 – Оценка понимания назначения приложения

Рисунок 3.12 – Оценка удобства форм, календаря, выбора дат и задач

Рисунок 3.13 – Общая оценка интерфейса приложения

В главе показано, как основные пользовательские сценарии реализованы через интерфейс, модели представления, сервисы, репозитории и системные API iOS. Автор выполнил ручную проверку предусмотренных техническим заданием функций и ограничений. Пользовательское исследование при этом рассматривается отдельно как предварительная оценка восприятия интерфейса.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы разработано iOS-приложение Luma для адаптивного планирования задач с учётом состояния пользователя. Работа выполнена для частного заказчика на основе предоставленных технических данных.

В ходе работы была проанализирована предметная область, рассмотрены существующие решения и определены основные пользовательские сценарии. Для описания структуры и поведения приложения построены UML-модели, спроектирована архитектура и выбраны средства разработки. Реализовано взаимодействие с системным календарём iOS, локальными уведомлениями, биометрической аутентификацией и Live Activities. Особое внимание уделено совместной работе отдельных функций: сведения о задачах, доступном времени и состоянии пользователя используются в рамках единого процесса планирования дня.

Поставленное задание выполнено: предусмотренные техническим заданием функциональные и нефункциональные требования реализованы и проверены автором вручную. Пользовательское исследование дало предварительную оценку восприятия интерфейса, но не являлось приёмочным испытанием. Дальнейшая работа может включать замену прототипной схемы проверки пароля, калибровку коэффициентов анализа дня и уточнение интерфейса по результатам практического использования.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Новикова Ю. Л., Аулова О. Ю., Быковская Е. С. Физиология сна и его влияние на психоэмоциональное состояние студентов [Электронный ресурс] // Вестник МГПУ. Серия «Естественные науки». – 2023. – № 2(50). – DOI: 10.25688/2076-9091.2023.50.2.04. – URL: https://iest-vestnik.mgpu.ru/wp-content/uploads/sites/5/2023/08/novikova-yu-l-aulova-o-yu-bykovskaya-e-s.pdf (дата обращения: 03.06.2026). – Текст : электронный.

2. Weiss H. M., Merlo K. L. Affect, Attention, and Episodic Performance [Электронный ресурс] // Current Directions in Psychological Science. – 2020. – Vol. 29, № 5. – P. 453–459. – DOI: 10.1177/0963721420949496. – URL: https://journals.sagepub.com/doi/10.1177/0963721420949496 (дата обращения: 03.06.2026). – Текст : электронный.

3. Aeon B., Faber A., Panaccio A. Does time management work? A meta-analysis [Электронный ресурс] // PLOS ONE. – 2021. – Vol. 16, № 1. – e0245066. – DOI: 10.1371/journal.pone.0245066. – URL: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0245066 (дата обращения: 03.06.2026). – Текст : электронный.

4. TickTick: A To-Do List and Calendar to Keep You Organized [Электронный ресурс] // TickTick. – URL: https://www.ticktick.com/ (дата обращения: 25.09.2026). – Текст : электронный.

5. Structured: Daily Planner [Электронный ресурс] // Structured. – URL: https://structured.app/ (дата обращения: 25.09.2026). – Текст : электронный.

6. How to Create, Edit & Delete Tasks [Электронный ресурс] // Structured Help Center. – URL: https://help.structured.app/en/articles/338050 (дата обращения: 25.09.2026). – Текст : электронный.

7. Simplify Task Management with Todoist [Электронный ресурс] // Todoist. – URL: https://www.todoist.com/task-management (дата обращения: 25.09.2026). – Текст : электронный.

8. The Fabulous: Build Better Habits & Achieve Your Goals [Электронный ресурс] // Fabulous. – URL: https://www.thefabulous.co/ (дата обращения: 25.09.2026). – Текст : электронный.

9. How Does Fabulous Work? [Электронный ресурс] // The Fabulous Help Center. – URL: https://help.thefabulous.co/en/support/solutions/articles/101000427430-how-does-fabulous-work- (дата обращения: 25.09.2026). – Текст : электронный.

10. UIKit [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/uikit (дата обращения: 26.09.2026). – Текст : электронный.

11. SwiftUI [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/swiftui (дата обращения: 26.09.2026). – Текст : электронный.

12. ActivityKit [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/activitykit (дата обращения: 26.09.2026). – Текст : электронный.

13. WidgetKit [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/widgetkit (дата обращения: 13.10.2026). – Текст : электронный.

14. Core Motion [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/coremotion (дата обращения: 26.09.2026). – Текст : электронный.

15. Xcode [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/xcode (дата обращения: 26.09.2026). – Текст : электронный.

16. The Swift Programming Language [Электронный ресурс] // Swift.org. – URL: https://www.swift.org/documentation/tspl/ (дата обращения: 26.09.2026). – Текст : электронный.

17. Git [Электронный ресурс] // Git Documentation. – URL: https://git-scm.com/docs/git (дата обращения: 26.09.2026). – Текст : электронный.

18. Core Data [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/coredata (дата обращения: 26.09.2026). – Текст : электронный.

19. Keychain services [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/security/keychain-services (дата обращения: 26.09.2026). – Текст : электронный.

20. Apple CryptoKit [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/cryptokit (дата обращения: 26.09.2026). – Текст : электронный.

21. EventKit [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/eventkit (дата обращения: 26.09.2026). – Текст : электронный.

22. User Notifications [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/usernotifications (дата обращения: 26.09.2026). – Текст : электронный.

23. Local Authentication [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/localauthentication (дата обращения: 26.09.2026). – Текст : электронный.

24. AVFoundation [Электронный ресурс] // Apple Developer Documentation. – URL: https://developer.apple.com/documentation/avfoundation (дата обращения: 26.09.2026). – Текст : электронный.

25. Fowler M. Presentation Model [Электронный ресурс] // martinfowler.com. – 2004. – URL: https://martinfowler.com/eaaDev/PresentationModel.html (дата обращения: 21.06.2026). – Текст : электронный.

26. Model-View-ViewModel (MVVM) [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm (дата обращения: 21.06.2026). – Текст : электронный.

27. Khanlou S. Coordinators Redux [Электронный ресурс] // Khanlou. – 2015. – URL: https://khanlou.com/2015/10/coordinators-redux/ (дата обращения: 21.06.2026). – Текст : электронный.

28. Martin R. C. The Clean Architecture [Электронный ресурс] // The Clean Code Blog. – 2012. – URL: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html (дата обращения: 21.06.2026). – Текст : электронный.

29. Apple Inc. Model-View-Controller [Электронный ресурс] // Apple Developer Documentation Archive. – 2018. – URL: https://developer.apple.com/library/archive/documentation/General/Conceptual/DevPedia-CocoaCore/MVC.html (дата обращения: 21.06.2026). – Текст : электронный.

30. Fowler M. Passive View [Электронный ресурс] // martinfowler.com. – 2006. – URL: https://martinfowler.com/eaaDev/PassiveScreen.html (дата обращения: 26.09.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А

Диаграммы последовательности ключевых сценариев

Рисунок А.1 – Авторизация с помощью Face ID

Рисунок А.2 – Интеграция с системным календарём iOS

Рисунок А.3 – Формирование статистики и обновление достижений

ПРИЛОЖЕНИЕ Б

Расчёт показателей анализа дня

Рисунок Б.1 – Расчёт оценки ресурса пользователя

Рисунок Б.2 – Подготовка задач и расчёт показателей трудоёмкости

Рисунок Б.3 – Расчёт фактической нагрузки, баланса и достоверности

Рисунок Б.4 – Формирование рекомендаций и результата анализа дня